

tf.compat.v1.depth_to_space Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/depth_to_space

DepthToSpace for tensors of type T.

Function Signatures

```
tf.compat.v1.depth_to_space( input, block_size, name=None,  
data_format='NHWC' )
```

Code Examples

```
tf.compat.v1.depth_to_space(  
input, block_size, name=None, data_format='NHWC'  
)
```

```
tf.compat.v1.depth_to_space(  
input, block_size, name=None, data_format='NHWC'  
)
```

```
x = [[[[[1, 2, 3, 4]]]]]
```

```
x = [[[[[1, 2, 3, 4]]]]]
```

```
[[[[[1], [2]],  
[3], [4]]]]]
```

```
[[[[[1], [2]],  
[3], [4]]]]]
```

```
x = [[[[[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]]]]]
```

```
x = [[[[[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]]]]]
```

Documentation

DepthToSpace for tensors of type T.

Rearranges data from depth into blocks of spatial data.

This is the reverse transformation of SpaceToDepth. More specifically, this op outputs a copy of the input tensor where values from the depth dimension are moved in spatial blocks to the height and width dimensions. The attr `block_size` indicates the input block size and how the data is moved.

- Chunks of data of size `block_size * block_size` from depth are rearranged

into non-overlapping blocks of size `block_size x block_size`

- The width of the output tensor is `input_depth * block_size`, whereas the

height is `input_height * block_size`.

- The Y, X coordinates within each block of the output image are determined

by the high order component of the input channel index.

- The depth of the input tensor must be divisible by

`block_size * block_size`.

The `data_format` attr specifies the layout of the input and output tensors with the following options:

"NHWC": [batch, height, width, channels]

"NCHW": [batch, channels, height, width]

"NCHW_VECT_C":

qint8 [batch, channels / 4, height, width, 4]

It is useful to consider the operation as transforming a 6-D Tensor.

e.g. for `data_format = NHWC`,

Each element in the input tensor can be specified via 6 coordinates,

ordered by decreasing memory layout significance as:

n, iY, iX, bY, bX, oC (where n =batch index, iX, iY means X or Y coordinates within the input image, bX, bY means coordinates within the output block, oC means output channels).

The output would be the input transposed to the following layout:

n, iY, bY, iX, bX, oC

This operation is useful for resizing the activations between convolutions (but keeping all data), e.g. instead of pooling. It is also useful for training purely convolutional models.

For example, given an input of shape $[1, 1, 1, 4]$, `data_format = "NHWC"` and `block_size = 2`:

This operation will output a tensor of shape $[1, 2, 2, 1]$:

Here, the input has a batch of 1 and each batch element has shape $[1, 1, 4]$, the corresponding output will have 2×2 elements and will have a depth of 1 channel ($1 = 4 / (\text{block_size} * \text{block_size})$).

The output element shape is $[2, 2, 1]$.

For an input tensor with larger depth, here of shape $[1, 1, 1, 12]$, e.g.

This operation, for block size of 2, will return the following tensor of shape $[1, 2, 2, 3]$

Similarly, for the following input of shape $[1, 2, 2, 4]$, and a block size of 2:

the operator will return the following tensor of shape $[1, 4, 4, 1]$:

Returns

